

RI402

Dizajn kompajlera

dr.sc. Edin Pjanić

Pregled predavanja

- Bottom-up parsiranje
 - Shift-reduce parser
- Yacc (bison) – generator parsera

Bottom-up parsiranje

- Opštiji od top-down metoda parsiranja, mada su neke ideje zajedničke.
- Pokriva veći dijapazon gramatika.
- Gramatiku nije potrebno pripremati na poseban način
 - Lijeva rekurzija i faktorisanje nije problem
- Efikasno kao i top-down
 - Generisanje tabela parsiranja složenije ali se radi pomoću alata.
- **Pristup**
 - Kao i kod top-down metoda, cilj je pronaći odgovarajuću derivaciju.
 - Korak po korak.
 - Počinje se od ulaznog stringa (sekvence tokena) i traže se koraci unazad do početnog simbola gramatike.
 - U svakom koraku derivacija ide unazad, tj. tijelo neke produkcije mijenjamo odgovarajućim neterminalom (vršimo *redukciju*).

Bottom-up parsiranje: Redukcija

- Za gramatiku:

$S \rightarrow E$

$E \rightarrow E + T$

$\rightarrow T$

$T \rightarrow T * \text{num}$

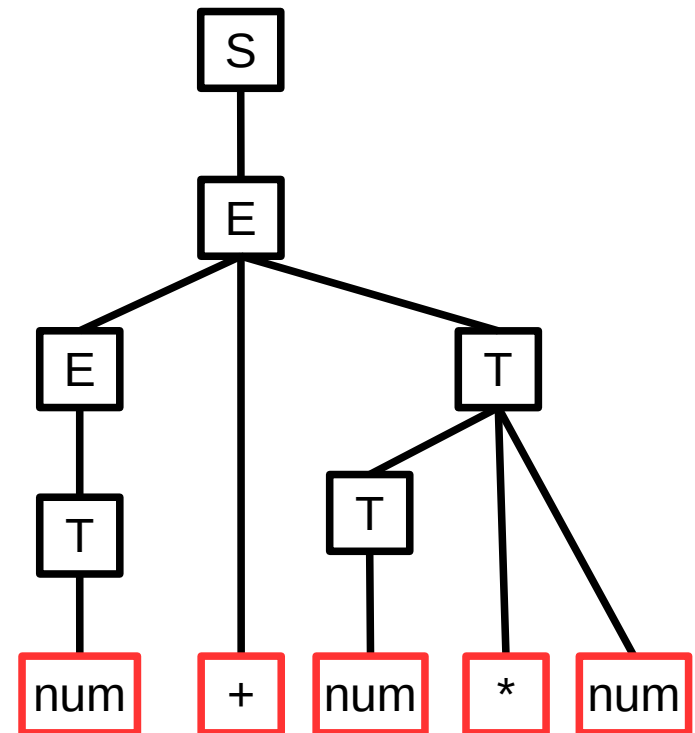
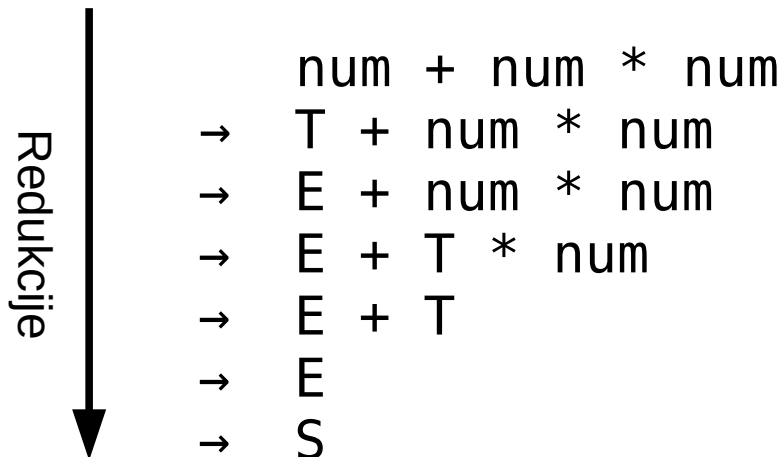
$\rightarrow \text{num}$

$\rightarrow (E)$

- I ulaznu sekvencu tokena:

$\text{num} + \text{num} * \text{num}$

Imaćemo:



Handle

Handle je podstring rečeničnog oblika koji odgovara tijelu nekog produkcijskog pravila.

Upravo redukcija handle-a predstavlja jedan korak u obrnutoj desnoj derivaciji.

Primjer sa prethodnog slajda:

Desni rečenični oblik

num + num * num
T + num * num
E + num * num
E + T * num
E + T
E

Handle

num
T
num
T * num
E + T
E

Produkcija korištena u redukciji

$T \rightarrow \text{num}$
 $E \rightarrow T$
 $T \rightarrow \text{num}$
 $T \rightarrow T * \text{num}$
 $E \rightarrow E + T$
 $S \rightarrow E$

Uočite da se desno od handle-a nalaze samo terminalni simboli.

!!! Da bi podstring bio handle, rečenični oblik nakon redukcije handle-a mora odgovarati prethodnom rečeničnom obliku desne derivacije u odnosu na trenutni rečenični oblik.

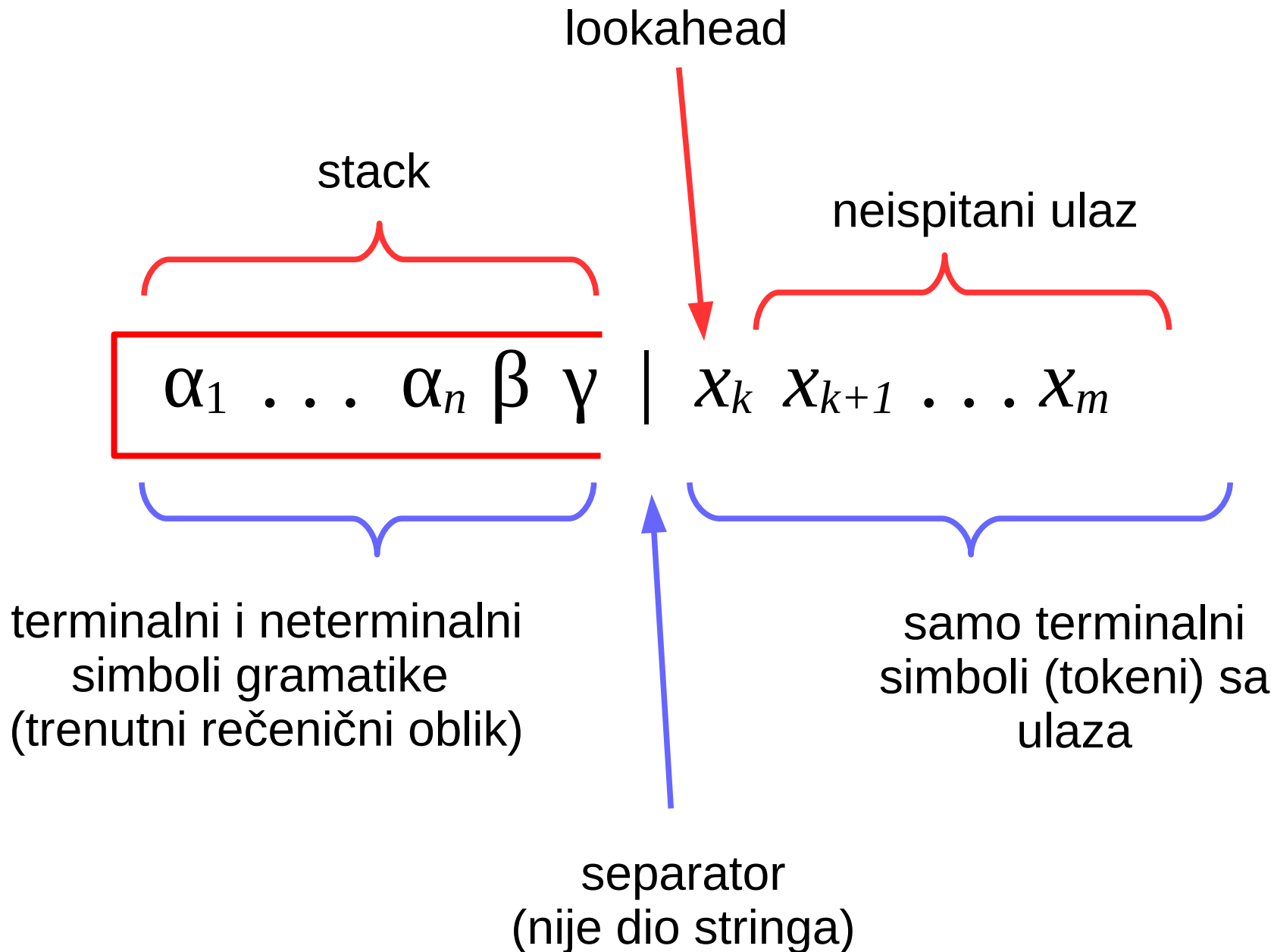
Shift-reduce parsiranje

- Vrsta bottom-up parsiranja.
- U pozadini je automat koji koristi pripremljene tranzicijske tabele za određivanje narednog stanja.
- Nema backtrackinga.
- Vrší jedan prolaz kroz ulaznu sekvencu tokena.
- Generiše desnu derivaciju.
- Naziv:
 - Reduce: redukcija
 - Shift: operacija uzimanja jednog tokena sa ulaza
- Za trenutni proizveden (redukovan) dio rečenice u implementaciji se koristi stack.

LR(k) parser

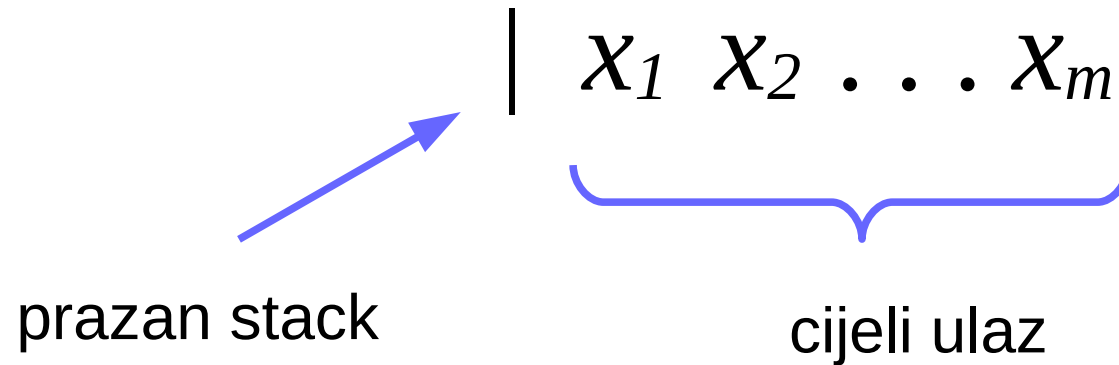
- Shift-reduce parser pripada porodici LR(k) parsera:
 - L – način skeniranja ulaza: “sLijeva udesno” bez backtrackinga
 - R – generiše desnu (Right) derivaciju
 - (k) – koristi najviše k tokena za *lookahead*
- Neke od vrsta LR parsera:
 - LR(k) – kanonički LR parser
 - LALR(k) – look-ahead LR parser
 - GLR – generalizovani LR parser
 - SLR – jednostavni (Simple) LR parser

Princip rada shift-reduce parsera

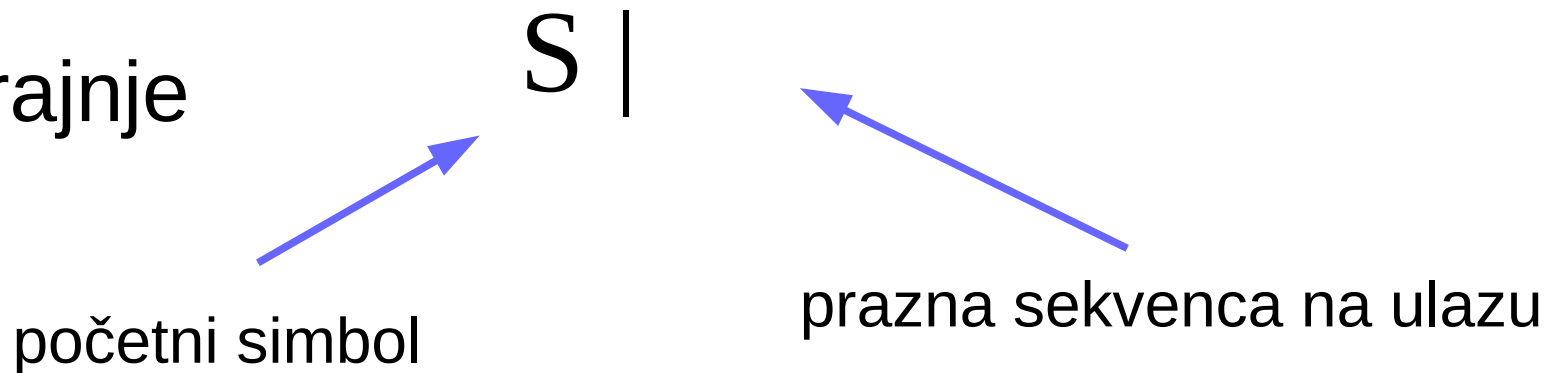


Početno i krajnje stanje parsera

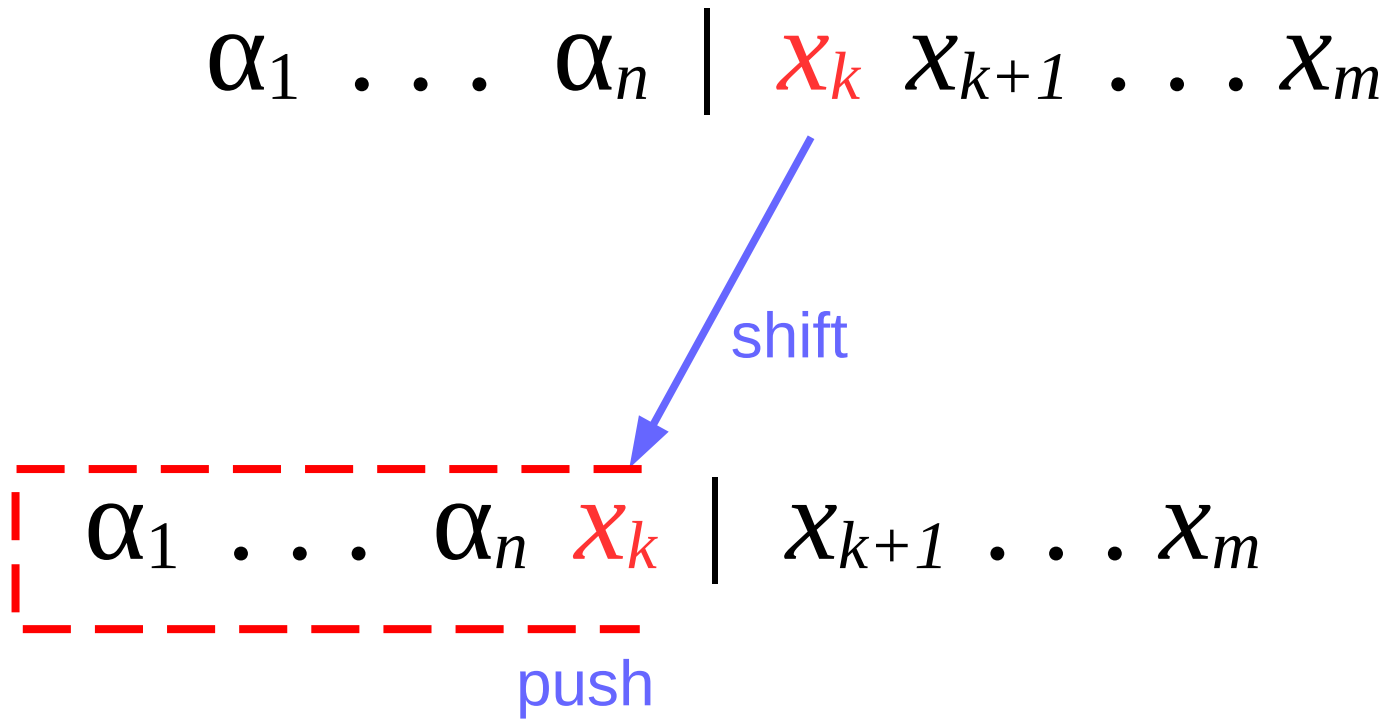
- Početno



- Krajnje



Operacija SHIFT – premještanje tokena na vrh steka



Operacija REDUCE – redukcija

$$\alpha_1 \ . \ . \ . \ \alpha_n \ \beta \ \gamma \mid X_k \ X_{k+1} \ . \ . \ . \ X_m$$

Redukuj $X \rightarrow \beta \ \gamma$
(uvažavajući lookahead,
tj. $\text{Follow}(X)$)

$$\alpha_1 \ . \ . \ . \ \alpha_n \ X \mid X_k \ X_{k+1} \ . \ . \ . \ X_m$$

Lookahead obezbjeđuje da se dobije prethodni rečenični oblik desne derivacije.

Ostale akcije

- Osim shift i reduce, moguće su još dvije akcije:
- “prihvati” - kada parser dođe u krajnje, prihvatajuće, stanje
- “greška” - postoji greška u kodu koji se parsira i potrebno je pozvati rutine za obradu greške
- Shift-reduce parser radi uz pomoć tabele stanja koja specificira koje je naredno stanje uvažavajući:
 - lookahead i
 - stanje na steku

Konflikti

- Prilikom parsiranja mogući su konflikti:
- Shift / reduce konflikt
 - Za slučaj da je moguće izvršiti i shift i reduce akciju
 - Obično je shift rješenje (standardno u generatorima parsera)
 - Moguće je definisati prioritet
- Reduce / reduce konflikt:
 - Za slučaj da postoje dvije produkcije za koje je moguće izvršiti redukciju simbola na steku
 - Ovo je nepoželjna situacija i znači da je gramatika višeznačna
 - Uglavnom se rješava promjenom gramatike
 - Generatori parsera obično uzimaju prvu produkciju koja je navedena u kodu

Shift-reduce parser: primjer parsiranja

| num + num * num \$ shift

num | + num * num \$ redukcija $T \rightarrow \text{num}$

T | + num * num \$ redukcija $E \rightarrow T$

E | + num * num \$ shift

E + | num * num \$ shift

E + num | * num \$ redukcija $T \rightarrow \text{num}$

E + T | * num \$ shift

E + T * | num \$ shift

E + T * num | \$ redukcija $T \rightarrow T * \text{num}$

E + T | \$ redukcija $E \rightarrow E + T$

E | \$ redukcija $S \rightarrow E$

S | \$ shift

S \$ | **prihvatanje**

(	$P \rightarrow S \$$
S	$\rightarrow E$
E	$\rightarrow E + T$
	$\rightarrow T$
T	$\rightarrow T * \text{num}$
	$\rightarrow \text{num}$
	$\rightarrow (E)$

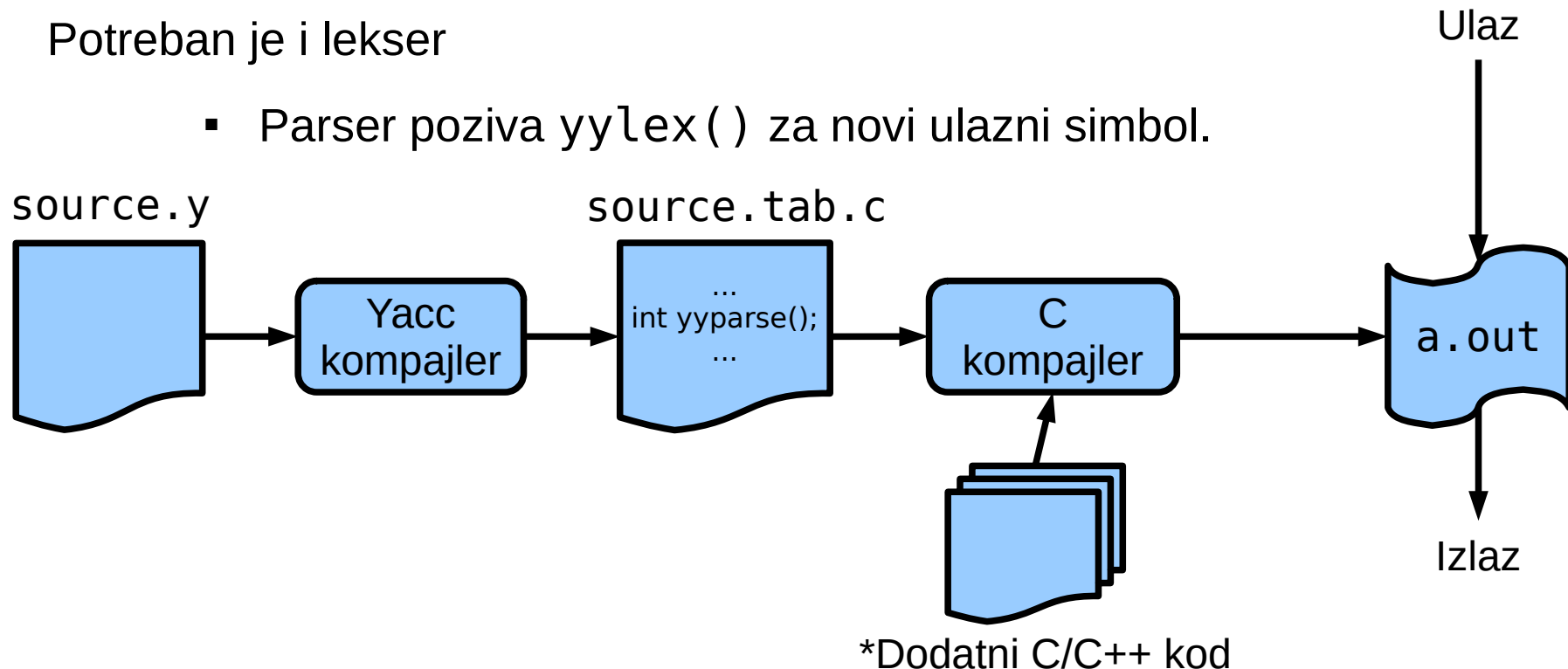
Follow(S) = { \$ }

Follow(E) = { +,), \$ }

Follow(T) = { +, *,), \$ }

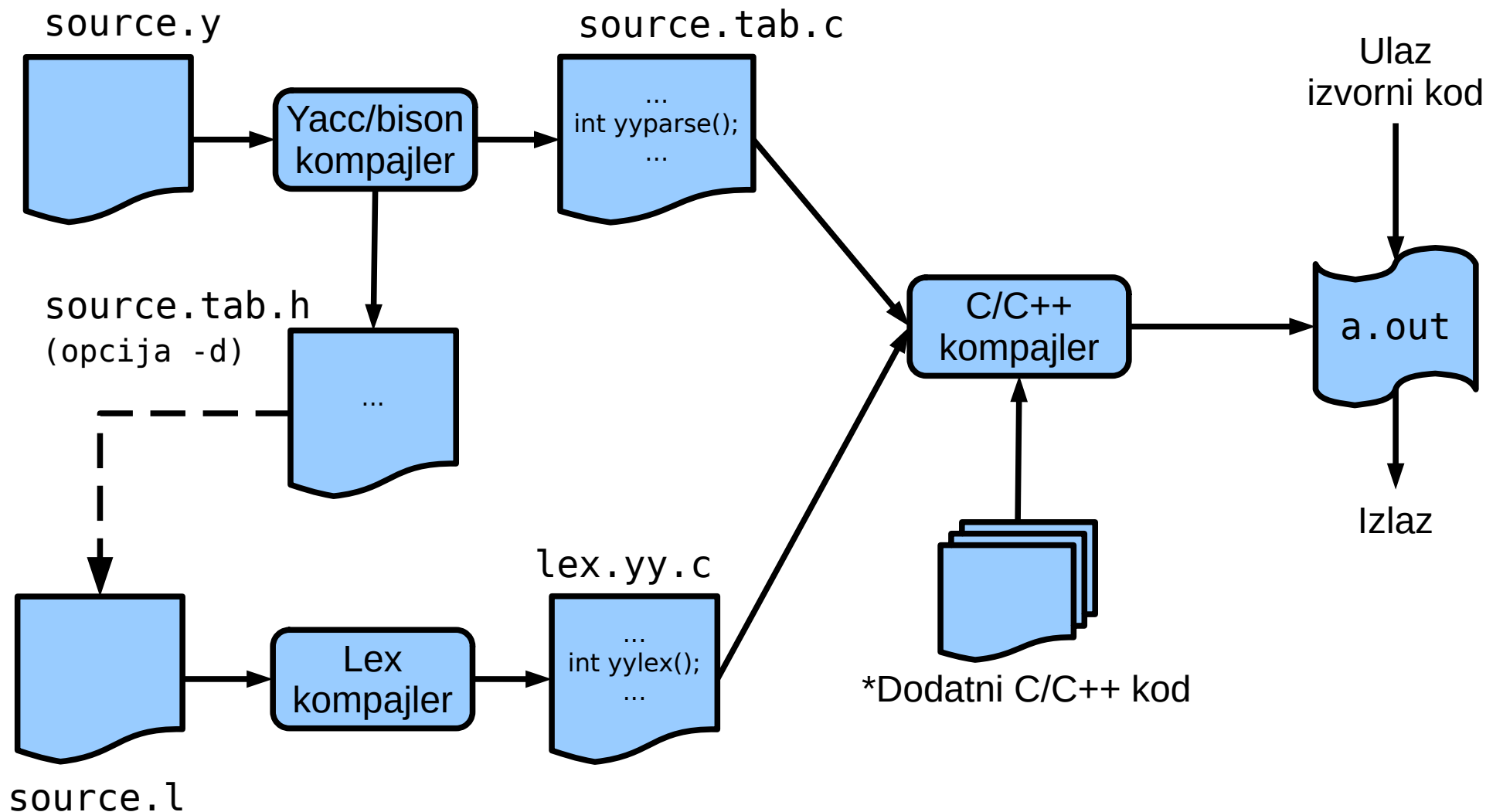
Yacc (bison)

- Alat za generisanje parsera (yet another compiler-compiler)
- Gramatika: Backus Naur Forma (BNF)
- Primarno radi kao LR(1) parser i eksperimentalno za druge LR algoritme
- Yacc kompajler proizvodi `srcname.tab.c` fajl
 - Simulira DFA
- Potreban je i lekser
 - Parser poziva `yylex()` za novi ulazni simbol.



Yacc - lex (bison-flex)

- Najčešći način korištenja yacc kompajlera



Struktura Yacc (bison) programa

`%{`

C definicije

(ovaj dio se kopira u srcname.tab.c)

`%}`

Definicije tokena

Definicije

`%%`

Gramatička pravila

`%%`

Pomoćne C funkcije

Definisanje gramatičkih pravila sa akcijama.

Format pravila:

`produkcija { akcija }`

Direktno se kopira u srcname.tab.c

Globalne varijable i funkcije

- `int yyparse()`
 - Funkcija koja vrši parsiranje.
- `int yylex()`
 - Funkcija kojom se dobija token. Mora se implementirati. (flex)
- `void yyerror(char *)`
 - Funkcija koju yacc poziva u slučaju greške. Implementirati.
- `main`
 - Ulazna tačka.
- `YYSTYPE`
 - Tip semantičke vrijednosti, default `int`
- `YYSTYPE yylval`
 - Semantička vrijednost simbola (globalna varijabla)

```
%{
#include <stdio.h>
int yylex(void);
void yyerror(const char *);
%}
%token INTEGER
```

```
%%
program:
    program expr '\n'
    | %empty
    ;
expr:
    INTEGER
    | expr '+' expr
    | expr '-' expr
    ;
```

```
%%
void yyerror(const char *s) {
    fprintf(stderr, "%s\n", s);
}
```

```
int main(void) {
    yyparse();
    return 0;
}
```

Primjer: jednostavni kalkulator

Parser će prijaviti grešku ako ulazni string ne odgovara specificiranoj gramatici.

Moramo imati lekser koji će imati funkciju yylex() i vraćati tokene.

```
P    → P E '\n'
      → ε
E    → int
      → E + E
      → E - E
```

Uočiti neterminalne simbole u specifikaciji gramatike:
program i **expr**.

Primjer: jednostavni kalkulator – lex fajl

```
%{  
    void yyerror(const char *);  
    #include "calc.tab.h"  
}%
```

```
%%  
[0-9]+      return INTEGER; ←  
[-+\n]      return *yytext; ←  
[ \t]       ; /* skip whitespace */  
.  
            yyerror("invalid character");  
%%
```

Kao token vraćamo
›INTEGER ili
›ASCII kod znaka.

```
int yywrap(void) {  
    return 1;  
}
```

```

%{
    #include <stdio.h>
    int ylex(void);
    void yyerror(const char *);
}%
%token INTEGER

%%
program:
    program expr '\n'    { printf("%d\n", $2); }
    | %empty
    ;
expr:
    INTEGER               { $$ = $1; }
    | expr '+' expr       { $$ = $1 + $3; }
    | expr '-' expr       { $$ = $1 - $3; }
    ;

%%
void yyerror(const char *s) {
    fprintf(stderr, "%s\n", s);
}
int main(void) {
    yyparse();
    return 0;
}

```

Primjer: jednostavni kalkulator 2

Izvršava operacije + i –
i ispisuje rezultat na
ekranu

Primjer: jednostavni kalkulator 2 – lex fajl

```
%{
    void yyerror(const char *);
    #include "calc.tab.h"
}%
%%
[0-9]+      {
              yylval = atoi(yytext);
              return INTEGER;
            }
[-+\n]      return *yytext;
[ \t]       ; /* skip whitespace */
.           yyerror("invalid character");
%%
int yywrap(void) {
    return 1;
}
```

yylval:
Vrijednost koja “odgovara”
leksemu tokena
(semantička vrijednost)



Razmisliti zašto ovakav kalkulator
ne daje rezultate koje očekujemo.
Kako riješiti taj problem?